

CHARLA TÉCNICA • 2026

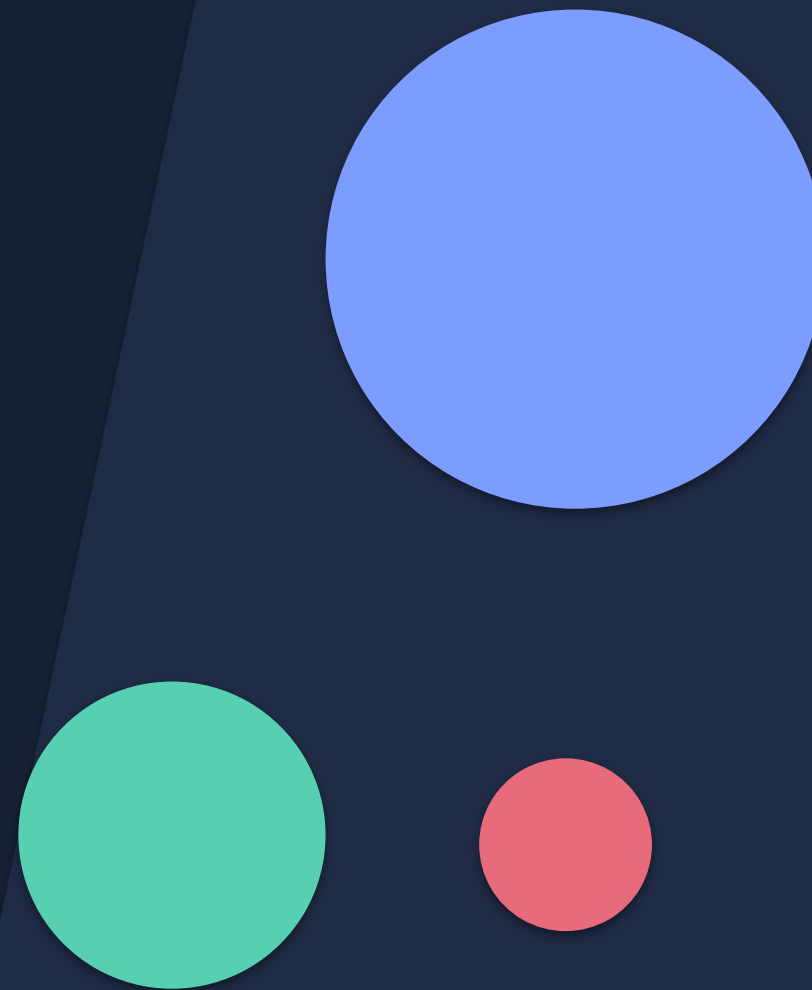
MemGPT

LLMs como sistemas operativos

Memoria jerárquica auto-gestionada por el agente

Implementación sobre LangGraph • Pavel Kim 2023 • arXiv:2310.08560

Máximo Fernández • maximofn.com



La ventana de contexto se llena

Síntoma

Un LLM tiene contexto finito.

Conversaciones largas, documentos grandes o tareas de varios días desbordan inevitablemente.

Cuando se trunca, el agente olvida decisiones, identidad y hechos clave.

La idea de Karpathy (2023)

“El LLM es la CPU; la ventana de contexto es la RAM.”

Y si es una CPU... necesita un sistema operativo: memoria virtual, planificador, llamadas al sistema.

MemGPT lo construye.

Memoria virtual para LLMs

Sistema operativo

- **CPU** Ejecuta instrucciones
- **RAM** Pequeña, rápida
- **Disco** Grande, lento
- **Paginación** Mueve datos RAM ↔ disco
- **Syscalls** Procesos piden al SO

MemGPT

- **LLM** Ejecuta turnos
- **Ventana de contexto** Lo que ve en cada inferencia
- **Almacenamiento externo** Recall / Archival / MemFS
- **Queue Manager** Expulsa y resume mensajes
- **Function calls** El LLM pide leer/escribir memoria

Tres piezas, un agente que se administra solo

01

Jerarquía de memoria

Rápida y pequeña (en contexto) + lenta e ilimitada (fuera).

02

Gestor de control de flujo

Decide cuándo invocar al LLM y cuándo expulsar y resumir mensajes.

03

Funciones expuestas al LLM

El propio modelo lee, escribe y busca en su memoria.

Punto clave: el LLM es auto-dirigido — decide qué guardar y qué buscar.

Cómo modifica MemGPT la ventana de contexto

System Instructions

Prompt + memGPT + funciones • solo lectura

Working Context

Bloques etiquetados editables (assistant / human / custom)

FIFO Queue

Slot 0: resumen recursivo • mensajes en orden cronológico

Agente normal

Window:

```
[system prompt]  
[user/agent turns...]
```

MemGPT

Window:

```
[system + memGPT prompt]  
[working context]  
[recursive summary]  
[user/agent + tool calls]
```

La libreta del agente, siempre visible

Bloque pequeño en el prompt, editable solo por el LLM vía tools.

- **Tamaño fijo por bloque** Presupuesto de tokens; no crece.
- **Read / Write** El LLM lo modifica; el harness, jamás.
- **Solo vía funciones** `core_memory_append` · `core_memory_replace`
- **Texto libre** Lenguaje natural, no JSON.
- **Persistente** Sobrevive entre sesiones.

Bloques típicos

assistant

Identidad del agente: nombre, personalidad, rol.

“Soy Sam, paciente y curiosa. No revelo que soy IA.”

human

Lo que el agente recuerda del usuario.

“Ana, vive en Madrid, le gusta el surf.”

custom

Bloques que el desarrollador añade.

`task_context`, `project_constraints`, `team_members...`

Cómo edita el LLM su libreta

core_memory_append

Añade información al final del bloque.

```
core_memory_append(  
    label="human",  
    content="Cumpleaños: 7 de febrero"  
)
```

core_memory_replace

Sustituye un fragmento (corrige, actualiza).

```
core_memory_replace(  
    label="human",  
    old="novio James",  
    new="ex-novio James"  
)
```

El LLM nunca escribe el bloque generando texto: solo a través de estas funciones.

El flujo en vivo de la conversación

0	Resumen recursivo	“Lo que pasó antes”, regenerado al hacer flush.
1	(user)	Hola, ¿te acuerdas dónde comimos?
2	(assistant)	tool_call: recall_memory_search(query='comida pacifica')
3	(tool_result)	[03/12] “Taco Bell junto a la playa”
4	(assistant)	Sí, en el Taco Bell de Pacífica.
5	(system)	Memory Pressure: contexto al 75%
...	(expulsados)	Se mueven a Recall Storage

↑ más antiguos (se expulsan)

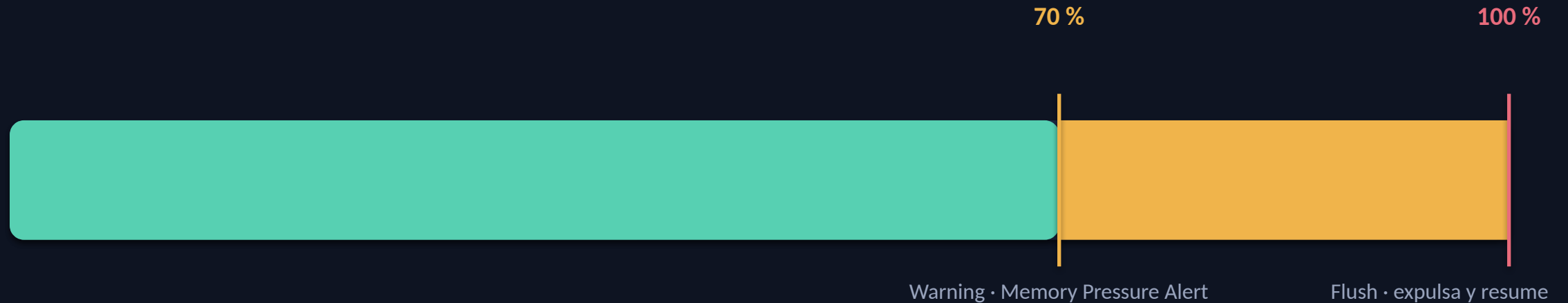
más recientes ↓

Entra en la FIFO

- Mensajes de usuario y agente
- Tool calls de MemGPT y otras tools
- Resultados de las funciones
- Mensajes del sistema (alertas, eventos)
- Sellos temporales para anclar al agente

Lo escribe el Queue Manager, no el LLM:
es un log automático.

70 % avisa, 100 % expulsada



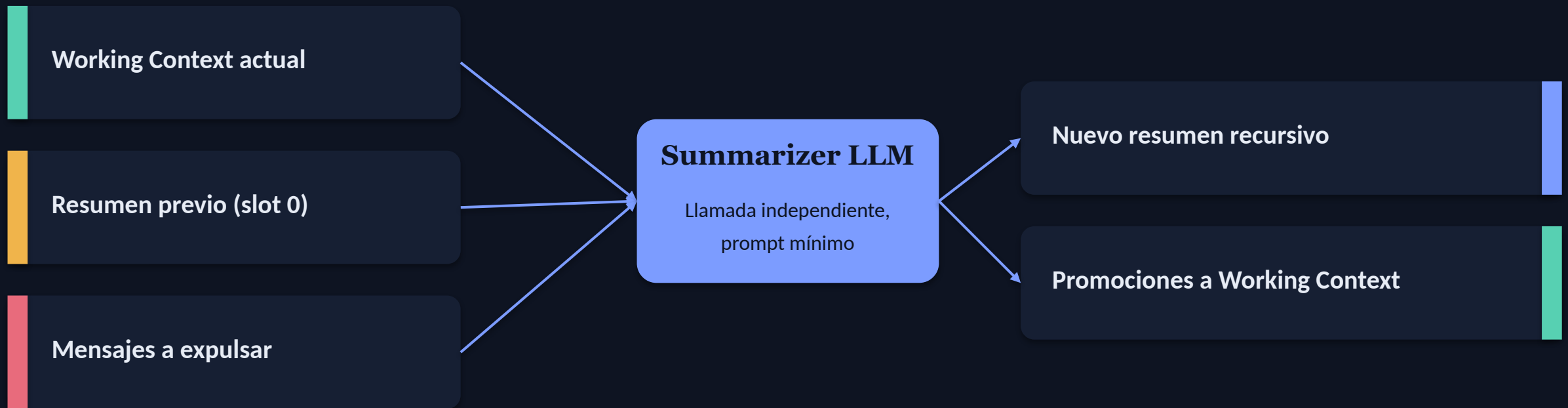
Warning (70 %)

Se inyecta una alerta en la FIFO.
El LLM consolida lo importante en
Working Context, Archival o MemFS.

Flush (100 %)

El Queue Manager expulsa los mensajes
más viejos (~50 %) y regenera el resumen.
Nada se pierde: vive en Recall Storage.

Una llamada al LLM fuera del bucle principal



¿Por qué fuera del bucle? Si el contexto está al 100 %, no cabe 'hazme un resumen' encima.
El summarizer recibe su propio prompt, sin las System Instructions del agente principal.

El log completo de la conversación

¿Qué es?

Historial completo, fuera del contexto.

Cada mensaje que entra a la FIFO se copia automáticamente a Recall en el mismo turno.

Cuando se hace flush, nada se pierde:
los mensajes expulsados ya están en Recall.

Cómo lo usa el LLM

```
recall_memory_search(query="...", page=N)
```

- **Solo lectura** el LLM busca, no inserta
- **Búsqueda semántica** embeddings + similitud
- **Paginada** evita 'lost in the middle'
- **Ilimitada** toda la historia
- **Persistente** entre sesiones

Lo que el agente decide guardar a largo plazo

¿Qué entra?

Texto arbitrario que el LLM elige persistir:

- resúmenes propios
- hechos sueltos
- documentos del usuario
- conclusiones de la sesión

Escritura explícita

```
archival_memory_insert(  
    content="..."  
)
```

Solo entra lo que el LLM decide insertar. Recall, no.

Lectura iterativa

```
archival_memory_search(  
    query="...", page=N  
)
```

Embeddings + paginación.
El LLM repite, reformula, pagina.

Mismo motor, distinto director

	RAG clásico	Archival Storage
Quién decide buscar	El pipeline, siempre	El LLM, solo si lo necesita
Iteración	Un disparo	Paginar, reformular, reintentar
Escritura	Corpus estático	El LLM puede archival_memory_insert
Resultado top-K	Injectado al prompt	El LLM lee y decide qué hacer

Archival = RAG (motor) + control del LLM (cuándo, cómo, cuántas veces) + escritura por el agente.

Memoria como filesystem versionado

```
/
├─ users/
│   └─ maximo/
│       └─ objetivos-q2.md
├─ projects/
│   └─ portafolio/
│       ├── decisiones.md
│       └─ notas.md
└─ learning/
    └─ spanish/
```

Tools de MemFS

```
memfs.create(path, content)
memfs.read(path)           memfs.write(path, content)
memfs.list(path)           memfs.move(src, dst)
memfs.grep(path, q)        memfs.history(path)
memfs.rollback(path, version)
```

Por qué versionado

- Proyectos largos en árbol jerárquico
- Trazabilidad: cómo evolucionó un fichero
- Rollback si el agente sobrescribe por error
- Pizarra compartida entre multi-agentes

Tres memorias que coexisten

	Working Context	Archival Storage	MemFS
Dónde vive	En la ventana de contexto	BD vectorial	Filesystem versionado
Tamaño	Pequeño (tokens)	Ilimitado	Ilimitado
Siempre visible	Sí	No (buscar)	No (navegar)
Búsqueda	Directa	Semántica	Path + grep
Versionado	No	No	Sí, tipo git
Coste por uso	Tokens en cada turno	Tool call	Tool call

Regla del agente

Working Context

Si el dato es...

Crítico, debe verse en cada turno.
Identidad, preferencias clave,
estado actual del usuario.

Ejemplo

```
"El usuario es Máximo,  
prefiere español, usa  
LangGraph + Astro."
```

Archival Storage

Si el dato es...

Voluminoso, no sabes cuándo lo necesitarás.
Búsqueda por significado a posteriori.

Ejemplo

```
"El usuario eligió pgvector  
porque..." →  
recuperable meses después.
```

MemFS

Si el dato es...

Estructurado, evoluciona en el tiempo,
necesitas historial.

Ejemplo

```
/projects/portafolio/  
decisiones.md  
+ memfs.history(...)
```


Qué pasa cuando llega un evento



El agente puede arrancarse solo

Wall-clock

El reloj del mundo dispara una acción.

Ejemplos

“Da los buenos días a las 09:00”
“Manda el correo en 30 minutos”
“Comprueba el estado cada 10 min”

Sleep-time agents

Cada N pasos del agente principal se activa un agente auxiliar.

Ejemplos

“Cada 10 turnos consolida memoria”
“Cada 20 mensajes revisa coherencia”
“Tras 50 turnos sin búsqueda, sugiere recuperar de archival”

request_heartbeat (LLMs antiguos) vs nativo

LEGACY · request_heartbeat=True

Argumento booleano en cualquier tool call.

Si está activo, MemGPT vuelve a invocar al LLM aunque éste hubiera 'parado'.

Evita que el modelo se rinda a medio buscar.

```
search_files(  
    query="MyClass",  
    request_heartbeat=True  
)
```

NATIVE · LLMs modernos

Si la respuesta contiene tool_calls, se ejecutan.

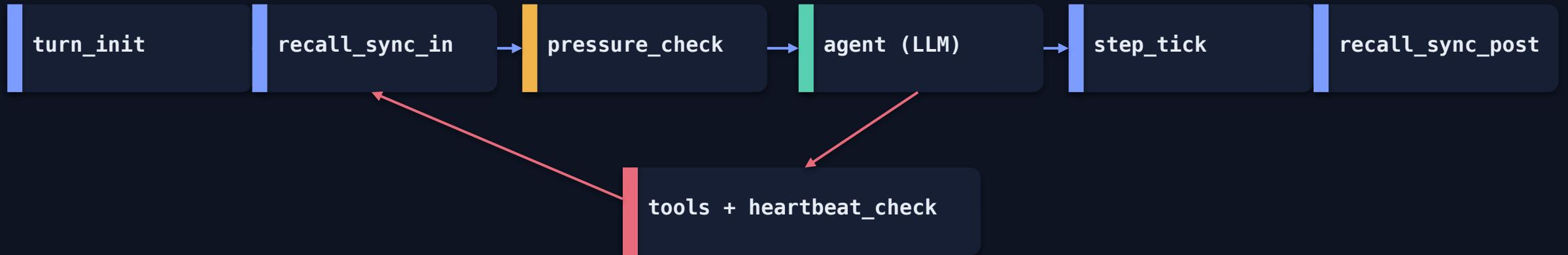
Si no, es el mensaje final al usuario.

No hace falta booleano: el LLM decide.

Protección contra bucles

- Límite de heartbeats por turno
- Timeout wall-clock
- Detección de tool calls repetidas sin progreso

MemGPT sobre LangGraph, desde cero



Stack

LangGraph

StateGraph + MemGPTState (Pydantic)

Postgres + pgvector

PostgresSaver + EventStore

Neo4j + Graphiti

Recall + Archival con embeddings

MemFS in-memory

9 tools versionadas (snapshot + SHA-1)

Lo que el LLM puede llamar

Core Memory

```
core_memory_append(label,  
content)
```

```
core_memory_replace(label  
, old, new)
```

Recall & Archival

```
recall_memory_search(quer  
y, page)
```

```
archival_memory_insert(co  
ntent)
```

```
archival_memory_search(qu  
ery, page)
```

MemFS

```
memfs.create / read /  
write
```

```
memfs.list / move /  
delete / grep
```

```
memfs.history / rollback
```

Comunicación

```
send_message(text)
```

Lo que aprendí construyéndolo

Bloques atómicos al expulsar

Nunca separar un AIMessage con tool_calls de sus ToolMessages.
Evita los bugs #111/#126 de langmem.

Recursive summary fuera de messages

Vive en su propio campo del estado y se inyecta como SystemMessage.
El reducer append-only no admite slots mutables.

parallel_tool_calls = False

Dos tools a la vez chocan en core_memory (un solo campo del state).
MemGPT está pensado para chaining, no fan-out.

Síncrono a propósito

El grafo se invoca con .invoke(); mezclar nodos async rompe con TypeError en LangGraph.

Qué te llevas de MemGPT

LLM como CPU

La ventana es RAM, lo externo es disco.

Memoria jerárquica

Working Context · FIFO · Recall · Archival · MemFS.

Auto-gestión

El propio modelo decide qué guardar, qué buscar y cuántas veces.

Control de flujo

Queue Manager + umbrales (70 % avisa, 100 % expulsa y resume).

Eventos automáticos

Wall-clock + sleep-time agents para consolidar sin usuario.



Gracias

Preguntas, comentarios, ideas.

Recursos

● Paper

[arXiv:2310.08560](https://arxiv.org/abs/2310.08560)

● Letta (sucesor)

github.com/letta-ai/letta

● Mi implementación

github.com/maximofn/memGPT

● Blog

maximofn.com